INGENIERÍA DEL SOFTWARE · ARQUITECTURA Y DISEÑO

Patrones de Diseño Aplicados al Proyecto Market

Documentación técnica de la refactorización mediante Factory Method, Iterator y Adapter

Este documento recoge el rediseño arquitectónico realizado sobre la aplicación **Market** con el objetivo de mejorar su modularidad, extensibilidad y desacoplamiento entre capas. Para ello se han aplicado tres patrones de diseño clásicos: **Factory Method** para la creación de la lógica de negocio, **Iterator** para el recorrido bidireccional de ofertas y **Adapter** para la visualización tabular de los productos de un vendedor.

1. Patrón Factory Method

a) Diagrama UML extendido y finalidad de las clases

El patrón **Factory Method** desacopla la capa de presentación (`ApplicationLauncher`) de la instanciación concreta de la lógica de negocio (`BLFacadeImplementation` o el servicio web remoto). La decisión sobre qué implementación instanciar se delega en la clase factoría `BLFactory` .

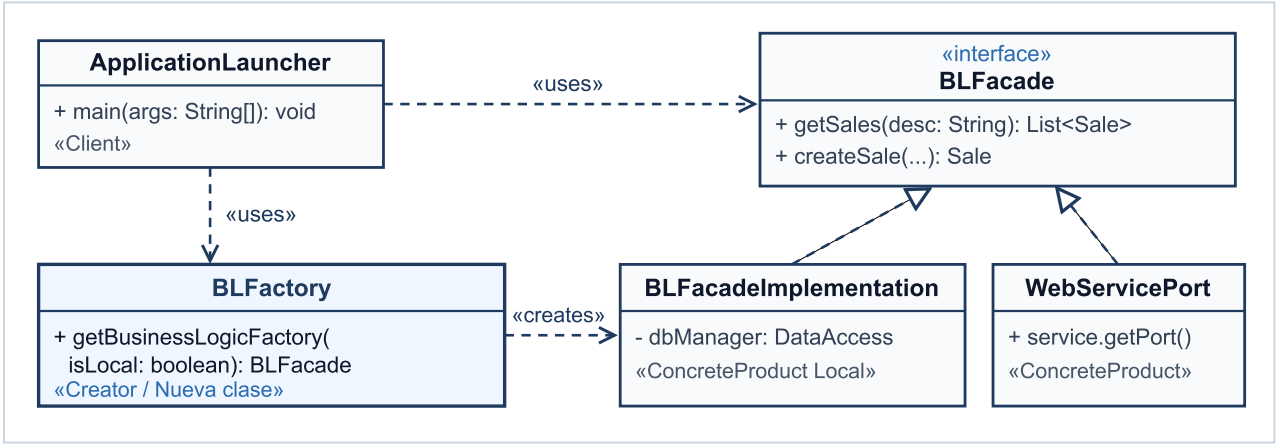


Figura 1. Diagrama UML del patrón Factory Method mostrando la centralización en BLFactory.

Clase / Interfaz	Rol en el Patrón	Finalidad y cambios realizados
BLFactory	Creator / Factory	**Nueva clase** en `businessLogic` . Encapsula la lógica condicional de creación de `BLFacade` (modo local frente a servicio web remoto WSDL).
BLFacade	Product	Interfaz común que define los servicios de la lógica de negocio consumidos por la interfaz gráfica.
BLFacadeImplementation	ConcreteProduct	Implementación local de la fachada que gestiona la persistencia a través de `DataAccess` .

Clase / Interfaz	Rol en el Patrón	Finalidad y cambios realizados
ApplicationLauncher	Client	**Modificada**. Elimina el bloque `if/else` de instanciación y delega la obtención de `BLFacade` a `BLFactory`.

b) Código creado y modificado

Se ha creado la clase `BLFactory` con el método fábrica `getBusinessLogicFactory(boolean isLocal)`, el cual consulta los parámetros de `ConfigXML` y devuelve la instancia correspondiente:

```
package businessLogic;

public class BLFactory {

    public BLFacade getBusinessLogicFactory(boolean isLocal) throws Exception {
        ConfigXML c = ConfigXML.getInstance();
        if (isLocal) {
            DataAccess da = new
DataAccess(c.getDataBaseOpenMode().equals("initialize"));
            return new BLFacadeImplementation(da);
        } else {
            String serviceName = "http://" + c.getBusinessLogicNode() + ":"
                    + c.getBusinessLogicPort() + "/ws/" + c.getBusinessLogicName()
+ "?wsdl";
            URL url = new URL(serviceName);
            QName qname = new QName("http://businessLogic/",
"BLFacadeImplementationService");
            Service service = Service.create(url, qname);
            return service.getPort(BLFacade.class);
        }
    }
}
```

En `ApplicationLauncher.java` se sustituye toda la lógica de construcción por una invocación directa a la factoría:

```
BLFactory factory = new BLFactory();
appFacadeInterface = factory.getBusinessLogicFactory(c.isBusinessLogicLocal());
MainGUI.setBussinessLogic(appFacadeInterface);
```

Diseño *Mockito-friendly*: Al centralizar la instanciación de `BLFacade` dentro de `BLFactory`, la arquitectura resulta altamente favorable para pruebas unitarias aisladas. Basta con incorporar un nuevo `ConcreteProduct` (un objeto simulado o *mock* creado con Mockito) devuelto por la factoría para verificar la interfaz gráfica o la lógica de control sin necesidad de conectar con la base de datos ni desplegar el servicio web.

2. Patrón Iterator

a) Diagrama UML extendido y finalidad de las clases

El patrón **Iterator** permite recorrer la colección de ofertas (`Sale`) de manera uniforme y en ambos sentidos (de principio a fin y de fin a principio) sin exponer la estructura interna de almacenamiento de la lógica de negocio.

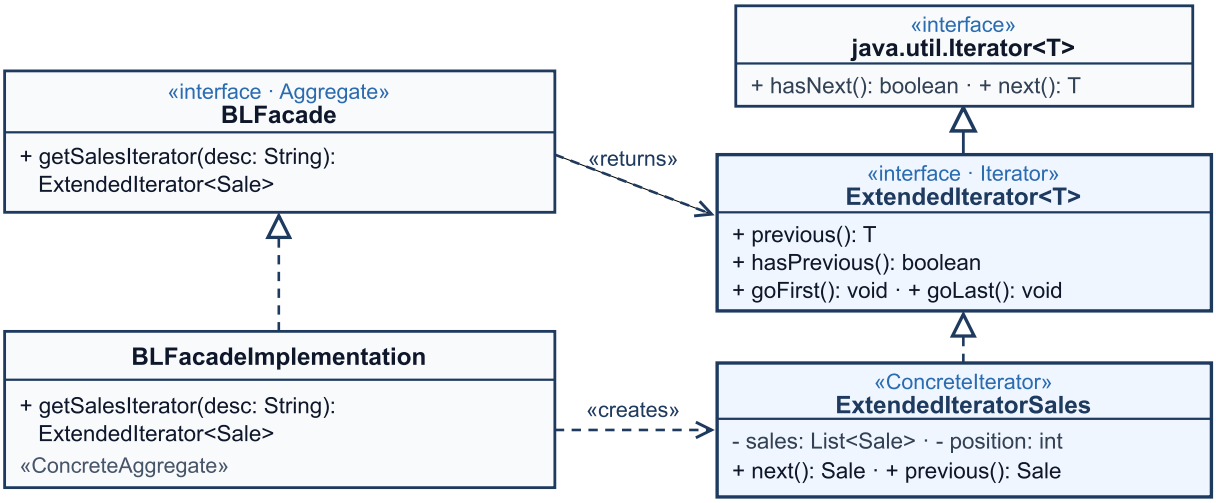


Figura 2. Diagrama UML del patrón Iterator extendido y su integración en la fachada de negocio.

Clase / Interfaz	Rol en el Patrón	Finalidad y cambios realizados
ExtendedIterator<T>	Iterator	**Nueva interfaz** en el paquete `iterator` . Extiende `java.util.Iterator<T>` añadiendo `previous()` , `hasPrevious()` , `goFirst()` y `goLast()` .
ExtendedIteratorSales	ConcreteIterator	**Nueva clase** que implementa `ExtendedIterator<Sale>` gestionando el índice de posición sobre una `List<Sale>` .
BLFacade / Implementation	Aggregate / ConcreteAggregate	**Modificadas**. Se incorpora el método `getSalesIterator(String desc)` , que reutiliza `getSales(desc)` y devuelve el iterador envuelto.

b) Código creado y modificado

En `ExtendedIteratorSales.java` se controla el cursor `position` para permitir avanzar o retroceder de forma segura sin desbordar los límites de la lista:

```
package iterator;

import java.util.List;
import domain.Sale;

public class ExtendedIteratorSales implements ExtendedIterator<Sale> {

    private List<Sale> sales;
    private int position;

    public ExtendedIteratorSales(List<Sale> sales) {
        this.sales = sales;
        this.position = 0;
    }

    @Override
    public boolean hasNext() {
        return sales != null && position >= 0 && position < sales.size();
    }

    @Override
    public Sale next() {
        Sale sale = sales.get(position);
        position++;
        return sale;
    }

    @Override
    public boolean hasPrevious() {
        return sales != null && position >= 0 && position < sales.size();
    }

    @Override
    public Sale previous() {
        Sale sale = sales.get(position);
        position--;
        return sale;
    }

    @Override
    public void goFirst() {
        position = 0;
    }

    @Override
    public void goLast() {
        if (sales != null && !sales.isEmpty()) {
            position = sales.size() - 1;
        } else {
            position = 0;
        }
    }
}
```

En `BLFacadeImplementation.java` se delega la consulta de datos al método existente `getSales(desc)` para evitar duplicidad de acceso a la base de datos:

```
@Override
public ExtendedIterator<Sale> getSalesIterator(String desc) {
    List<Sale> sales = getSales(desc);
    return new ExtendedIteratorSales(sales);
}
```

c) Captura de ejecución del patrón Iterator

Al ejecutar la clase de prueba `MainIterator` sobre la base de datos del proyecto, se verifica el recorrido inverso desde el último elemento (`goLast()`) y el recorrido directo desde el primero (`goFirst()`):

```
----- FROM LAST TO FIRST -----
8;sukaldeko mahaia;45.0
7;polar kilor erlojua;30.0
6;orbea mendiko bizikleta;225.0
5;iphone 17;400.0
4;imac 27;200.0
3;samsung 42" telebista;1.0
2;salomon mendiko botak;2.0
1;futbol baloia;2.0

----- FROM FIRST TO LAST -----
1;futbol baloia;2.0
2;salomon mendiko botak;2.0
3;samsung 42" telebista;1.0
4;imac 27;200.0
5;iphone 17;400.0
6;orbea mendiko bizikleta;225.0
7;polar kilor erlojua;30.0
8;sukaldeko mahaia;45.0
```

3. Patrón Adapter

a) Diagrama UML extendido y finalidad de las clases

El patrón **Adapter** permite integrar los datos de un vendedor (`Seller`) dentro de un componente visual Swing `JTable` sin modificar la arquitectura del dominio. La clase `SellerAdapter` hereda de `AbstractTableModel` y traduce la lista de ofertas en filas y columnas tabulares.

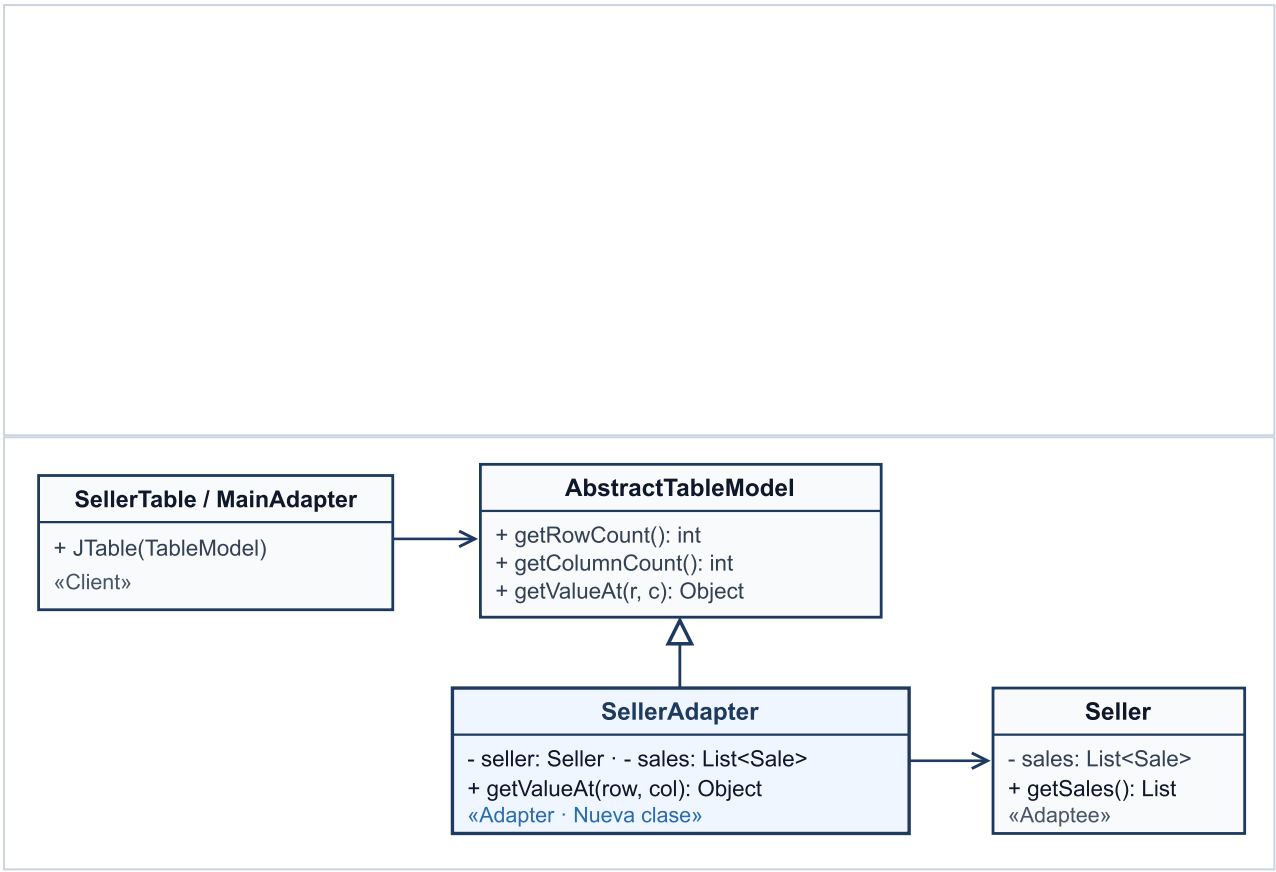


Figura 3. Diagrama UML del patrón Adapter entre Seller y el modelo de tabla Swing.

Clase	Rol en el Patrón	Finalidad y cambios realizados
SellerTable / MainAdapter	Client	Interfaz Swing con `JTable` que consume un `TableModel` para renderizar filas y columnas sin conocer la clase `Seller`.
AbstractTableModel	Target	Clase abstracta de Swing que define el contrato estándar requerido por `JTable`.
SellerAdapter	Adapter	**Nueva clase** en `adapter`. Extiende `AbstractTableModel` y traduce cada `Sale` del vendedor en tres columnas: `title`, `description` y `price`.
Seller	Adaptee	**Modificada**. Se añade el método público `getSales()` para exponer la lista de ofertas asociadas al vendedor.

b) Código creado y modificado

En `Seller.java` se incorpora el método de acceso `getSales()`, y en `SellerAdapter.java` se sobrescriben los métodos de consulta tabular para mapear los atributos de cada oferta:

```
package adapter;

import java.util.ArrayList;
import java.util.List;
import javax.swing.table.AbstractTableModel;
import domain.Sale;
import domain.Seller;

public class SellerAdapter extends AbstractTableModel {

    private Seller seller;
    private List<Sale> sales;
    private String[] columnNames = {"title", "description", "price"};

    public SellerAdapter(Seller seller) {
        this.seller = seller;
        if (seller != null && seller.getSales() != null) {
            this.sales = new ArrayList<>(seller.getSales());
        } else {
            this.sales = new ArrayList<>();
        }
    }

    @Override
    public int getRowCount() {
        return sales.size();
    }

    @Override
    public int getColumnCount() {
        return columnNames.length;
    }

    @Override
    public String getColumnName(int column) {
        return columnNames[column];
    }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        Sale sale = sales.get(rowIndex);
        switch (columnIndex) {
            case 0:
                return sale.getTitle();
            case 1:
                return sale.getDescription();
            case 2:
                return sale.getPrice();
            default:
                return null;
        }
    }
}
```

c) Captura de ejecución del patrón Adapter

La siguiente captura muestra la ventana Swing (`MainAdapter`) cargando dinámicamente el modelo `SellerAdapter` con las ofertas asociadas al vendedor *Aitor Fernandez*:

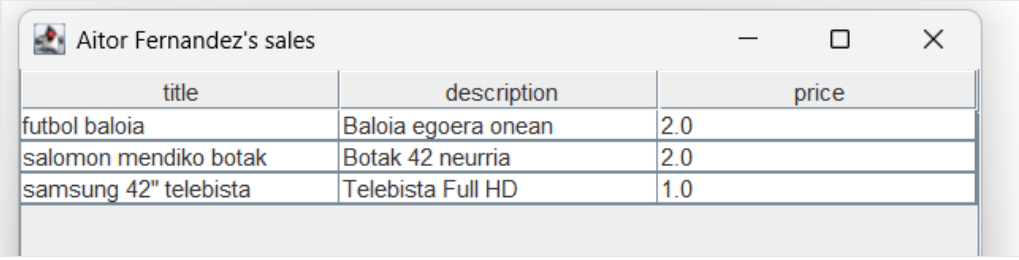


Figura 4. Ejecución de MainAdapter mostrando las ofertas del vendedor mediante SellerAdapter.

4. Declaración sobre el Uso de Inteligencia Artificial

> **Alcance del uso de herramientas de IA:** Se hace constar expresamente que el uso de inteligencia artificial en esta entrega se ha limitado única y exclusivamente a la maquetación, estructuración visual y presentación formal de este documento de memoria. Toda la arquitectura de software, el diseño de los patrones, la implementación del código fuente en Java, las pruebas de ejecución y la lógica de negocio del proyecto han sido desarrollados íntegramente por el autor sin generación automática de código mediante IA.